

CMP-4008Y Coursework 2 - Arcade System and Simulation

100484263 (exh24tdu)

Tue, 15 Apr 2025 12:13

PDF prepared using pass-cli-server version 1.3.1 running on Linux 5.4.0-200-generic (amd64).

☒ I agree that by submitting a PDF generated by PASS I am confirming that I have checked the PDF and that it correctly represents my submission.



Contents

Simulation.java	2
ArcadeGame.java	7
CabinetGame.java	8
ActiveGame.java	10
VirtualRealityGame.java	11
Customer.java	13
Arcade.java	17
AgeLimitException.java	23
Equipment.java	24
IncorrectIdException.java	25
InsufficientBalanceException.java	26
InvalidCustomerException.java	27
LevelOfDiscount.java	28
Application	29
Compiler Invocation	29
Compiler Messages	29
Application Invocation	29
Messages to STDOUT	29
Messages to STDERR	35

Simulation.java

```
1  import java.io.*;
   import java.util.*;
3
   public class Simulation {
5
       public static Arcade initialiseArcade(String arcadeName, File gamesFile, File
           customerFile) throws FileNotFoundException
7       {
           Arcade arcade = new Arcade(arcadeName);
           BufferedReader cReader = new BufferedReader(new FileReader(customerFile));
           BufferedReader gReader = new BufferedReader(new FileReader(gamesFile));
           String str;
           int line = 1;
           try
           {
15               while((str = cReader.readLine()) != null) // while loop used to go through
                   each row and read them
               {
17                   String[] customerInfo = str.split("#"); //splitting row by using # as
                       regex
19
                   String id = customerInfo[0].trim();
                   String name = customerInfo[1].trim();
21                   int balance = Integer.parseInt(customerInfo[2].trim());
                   int age = Integer.parseInt(customerInfo[3].trim());
23                   int numbOfElements = 4; //this variable is needed to check how many
                       elements does array contain if it is equal to this variable
                   // then it means there is level of discount missing which means it is
                       none
25                   if(numbOfElements == customerInfo.length)
                       {
27                       LevelOfDiscount discountLVL = LevelOfDiscount.NONE;
                       Customer customer = new Customer(id, name, balance, age, discountLVL)
                           ;
29                       arcade.addCustomer(customer);
                       }
31                   else
                       {
33                       String discountString = customerInfo[4].trim();
                       LevelOfDiscount discountLVL = LevelOfDiscount.valueOf(
                           discountString);
35                       Customer customer = new Customer(id, name, balance, age, discountLVL)
                           ;
37                       arcade.addCustomer(customer);
                       }
39
                   line++;
               }
41
           while ((str = gReader.readLine()) != null)
           {
43               String[] gamesInfo = str.split("@");
45
47               String id = gamesInfo[0].trim();
               String name = gamesInfo[1].trim();
               String type = gamesInfo[2].trim();
49               int price = Integer.parseInt(gamesInfo[3].trim());
               if(type.equals("cabinet"))
               {
51                   String reward = gamesInfo[4].trim();
53                   if(reward.equals("yes"))
```

```

55         {
56             boolean hasReward = true;
57             CabinetGame game = new CabinetGame(name, id, price, hasReward);
58             arcade.addArcadeGame(game);
59         }
60         else
61         {
62             boolean hasReward = false;
63             CabinetGame game = new CabinetGame(name, id, price, hasReward);
64             arcade.addArcadeGame(game);
65         }
66     }
67     else if(type.equals("active"))
68     {
69         int ageLimit = Integer.parseInt(gamesInfo[4].trim());
70         ActiveGame game = new ActiveGame(name, id, price, ageLimit);
71         arcade.addArcadeGame(game);
72     }
73     else
74     {
75         int ageLimit = Integer.parseInt(gamesInfo[4].trim());
76         String equipment = gamesInfo[5];
77         if(equipment.equals("headsetAndController"))
78         {
79             Equipment equipmentType = Equipment.HEADSET_AND_CONTROLLER;
80             VirtualRealityGame game = new VirtualRealityGame(name, id,
81                 price, ageLimit, equipmentType);
82             arcade.addArcadeGame(game);
83         }
84         else if(equipment.equals("headsetOnly"))
85         {
86             Equipment equipmentType = Equipment.HEADSET_ONLY;
87             VirtualRealityGame game = new VirtualRealityGame(name, id,
88                 price, ageLimit, equipmentType);
89             arcade.addArcadeGame(game);
90         }
91         else
92         {
93             Equipment equipmentType = Equipment.BODY_TRACKING_SUIT;
94             VirtualRealityGame game = new VirtualRealityGame(name, id,
95                 price, ageLimit, equipmentType);
96             arcade.addArcadeGame(game);
97         }
98     }
99 }
100 catch (IOException e)
101 {
102     e.printStackTrace();
103 }
104 return arcade;
105 }
106
107 public static void simulateFun(Arcade arcade ,File tranFile) throws
108     FileNotFoundException
109 {
110     BufferedReader tReader = new BufferedReader(new FileReader(tranFile));
111     String str;
112     int line = 1;

```

```

113     try
114     {
115         while((str = tReader.readLine()) != null)//while loop is used to check
            every single row and it has been divided on three branching parts
            where first one checks and performs transaction to allow customer to
            play
        {
117
118
119         String[] transactions = str.split(",");//splitting each row on
            certain amount of elements based on row itself
120         String action = transactions[0].trim();
121         String custID = transactions[1].trim();
122         if(action.equals("PLAY"))
123         {
124             try
125             {
126                 String gameId = transactions[2].trim();
127                 String peakStatus = transactions[3].trim();
128                 boolean peak;
129                 if(peakStatus.equals("PEAK"))
130                 {
131                     peak = true;
132                 }
133                 else
134                 {
135                     peak = false;
136                 }
137
138                 boolean result = arcade.processTransaction(custID,gameID,peak
139                     );
140                 if(result)
141                 {
142                     String getName = arcade.getCustomer(custID).
143                         getCustomerName();
144                     System.out.println(getName + ", you have successfully
145                         paid for the game. Have fun!");
146                 }
147             }
148             catch (IncorrectIdException e)
149             {
150                 System.out.println("Wrong format of id: " + e.getMessage());
151             }
152         }
153         else if (action.equals("NEW_CUSTOMER"))//The second part is about
            identifying the type of row to decide where a new customer should
            be added to the system."
154         {
155             String name = transactions[2].trim();
156             int numbOfElements = 6;//same principle used as in
            initialiseArcade method
157             if(numbOfElements == transactions.length)
158             {
159                 String discountLVL = transactions[3].trim();
160
161                 LevelOfDiscount level = LevelOfDiscount.valueOf(discountLVL);
162                 int balance = Integer.parseInt(transactions[4].trim());
163                 int age = Integer.parseInt(transactions[5].trim());
164                 Customer customer = new Customer(custID,name,balance,age,

```

```

        level);
        arcade.addCustomer(customer);
        checkForSuccess(arcade, custID);
    }
    else
    {
        int balance = Integer.parseInt(transactions[3].trim());
        int age = Integer.parseInt(transactions[4].trim());
        LevelOfDiscount discountLVL = LevelOfDiscount.NONE;
        Customer customer = new Customer(custID, name, balance, age,
            discountLVL);
        arcade.addCustomer(customer);
        checkForSuccess(arcade, custID);
    }
}
else //And if first element of the array starts with ADD_FUNDS this
parts performing adding money to the customer balance
{
    int fee = Integer.parseInt(transactions[2].trim());
    Customer customer = arcade.getCustomer(custID);
    try
    {
        int oldBalance = customer.getAccBalance();
        customer.addFunds(fee);
        if(oldBalance < customer.getAccBalance() )
        {
            System.out.println("You have successfully added money to
your balance");
        }
        else
        {
            System.out.println("Money haven't been added,ask for
assistance");
        }
    }
    catch (NullPointerException e)
    {
        System.out.println("Failed to add funds");
    }
}
line++;
}
System.out.println( "\n" + "The richest customer is " + arcade.
findRichestCustomer());
System.out.println("\n"+ "The median game price is: " + arcade.
getMedianGamePrice());
System.out.println("\n"+ "Cabinet/Active/Virtual games: " + Arrays.
toString(arcade.countArcadeGames()));
Arcade.printCorporateJargon();
System.out.println();

}
catch (IOException e)
{
    e.printStackTrace();
}

}

```

```
221 public static void checkForSuccess(Arcade arcade, String custID) // I have
    created this method just to avoid repetition of the code.
    // This method checks whether adding customer to the customers ArrayList is
    successful or not
223 {
    boolean existInsystem = false;
225 for (int i = 0; i < arcade.customers.size(); i++)
    {
227         if(arcade.customers.get(i).getCustomerID().equals(custID))//Loop through
            the array of customers until custID is equal to the id of one of the
            customers
            {
229                 existInsystem = true;
            }
231     }
    if(existInsystem)
233     {
        System.out.println("Customer has successfully been added to the system");
235     }
    else
237     {
        System.out.println("Customer haven't been added,ask for assistance");
239     }
}

241 public static void main(String[] args)
243 {
    File custFile = new File("customers.txt");
245 File gameFile =new File("games.txt");
    File trFile = new File("transactions.txt");
247 try
    {
249         Arcade passedArcade = initialiseArcade("Arcade of fun ",gameFile,custFile
            );
            simulateFun(passedArcade,trFile);
251         System.out.println(passedArcade);

253     }
    catch (IOException e)
255     {
        e.printStackTrace();
257     }
}

259 }
```

ArcadeGame.java

```

2  abstract public class ArcadeGame implements Comparable<ArcadeGame>
{
4      protected String name;
      protected String id;
6      protected int price;
      protected double newPrice;
8
      public ArcadeGame(String name,String id, int price)
10     {
          this.name = name;
12         this.id = id;
          this.price = price;
14         if(id.length() != 10) //Checking whether id is 10 characters long
            {
16             throw new IncorrectIdException("Id should be 10 characters long!");
            }
18     }

20     public String getName()
        {
22         return name;
        }
24     public String getID()
        {
26         return id;
        }
28     public int getPrice()
        {
30         return price;
        }
32     public int compareTo(ArcadeGame other)
        {
34         return Integer.compare(this.price,other.price);//The compareTo method is used
            to perform sorting on an ArrayList of games
        }
36

38     public abstract int calculatePrice(boolean peak);

40     // public String toString()
41     // {
42     //     return "The " + name + " has id: " + id + " and its price is " + price;
43     // }
44
45     // public static void main(String[] args)
46     // {
47     //     ArcadeGame obj = new ArcadeGame("race","1003238488",1);
48     //     System.out.println(obj);
49     //     ArcadeGame obj1 = new ArcadeGame("race","100323848",1);
50     // }
51     // }
52     //The main method is commented out because i used it for testing to check whether
        the exception is thrown
        // and whether the price is shown in pence
54 }

```

CabinetGame.java

```

public class CabinetGame extends ArcadeGame {
2     private boolean hasReward;

4     public CabinetGame(String name,String id,int price,boolean hasReward)
    {
6         super(name,id,price);
        this.hasReward = hasReward;
8         if(!id.startsWith("C"))
        {
10             throw new IncorrectIdException("The id should start from the letter C");
        }
12     }
    public boolean getHasReward()
14     {
        return hasReward;
16     }
    public int calculatePrice(boolean peak)//overridden calculatePrice method
18     {
        if(peak == true)// if it is peak time then price stays the same
20         {
            newPrice = price;
22             return (int) newPrice;
        }
24         else if (peak == false && hasReward == false)//if it is not peak time and
            there is no reward price gets 50% discount
        {
26             newPrice = price * 0.5;
            return (int) newPrice;
28         }
        else if (peak == false && hasReward == true)//This time if peak time and
            there is reward price gets 20% discount
30         {
            newPrice = price * 0.8;
32             return (int) newPrice;
        }
34         else//in other cases price stays the same
        {
36             newPrice = price;
            return (int) newPrice;
38         }
    }

40     public String toString()
    {
42         return "\n" + "\n" + "Name: " + name + "\n" + "Id: " + id + "\n" + "Price: " +
            price + "p \n" + "Status of the reward: : " + hasReward;
44     }

46     // public static void main(String[] args)
    // {
48     ////      CabinetGame cabGame = new CabinetGame("Pac-Man","C003322111",6,true);
    ////      System.out.println(cabGame.calculatePrice(false));
50     ////      System.out.println(cabGame);
    //      CabinetGame cabinetGame = new CabinetGame("aafaf","C000111222",500,true);
52     //      System.out.println(cabinetGame.toString());
    //  }

54     //Exception works well,if first character of id is not a letter C then exception
    // appears.The same with number of characters.
56     // Also,price is returned properly according to the discounts

```


58 }

ActiveGame.java

```

public class ActiveGame extends ArcadeGame {
2   protected int minAge;

4   public ActiveGame(String name,String id,int price,int minAge)
   {
6       super(name,id,price);
       this.minAge = minAge;
8       if(id.length() != 10 || !id.startsWith("A"))
       {
10          throw new IncorrectIdException("The id should start from the letter A
              because it is Active Game");
        }
12    }
    public int getMinAge()
14    {
        return minAge;
16    }
    public int calculatePrice(boolean peak) //same principle used as in Cabin game
        but without 50% discount and without rewards regardless age limit
18    {
        if(peak == true)
20        {
            newPrice = price;
22            return (int) newPrice;
        }
24        else if (peak == false)
        {
26            newPrice = price * 0.8;
            return (int) newPrice;
28        }
        else
30        {
            newPrice = price;
32            return (int) newPrice;
        }
34    }

    public String toString()
    {
36        //      return "Price of the game " + name + " with id " + id + " is: " + (int)
newPrice + "p and minimum age to play the game is: " + minAge;
        return "\n"+ "\n" + "Name: " + name + "\n" + "Id: " + id + "\n" + "Price: "
            + price + "p \n" + "Minimum age:" + minAge;
40    }

42    //      public static void main(String[] args)
    //      {
44        //          ActiveGame actGame = new ActiveGame("Air Hockey","A003322111",5,12);
        //          System.out.println(actGame.calculatePrice(true));
46        //          System.out.println(actGame);
        //      }

48    //I have tested the ActiveGame class the same way as cabinetGame class
50 }

```

VirtualRealityGame.java

```

public class VirtualRealityGame extends ActiveGame{
2  //    public enum Equipment {HEADSET_ONLY, HEADSET_AND_CONTROLLER, BODY_TRACKING_SUIT};
    private Equipment vrEquipment;

4
    public VirtualRealityGame (String name, String id, int price, int minAge, Equipment
        vrEquipment)
6    {
        super(name, id, price, minAge);
8        this.vrEquipment = vrEquipment;
        if(id.length() != 10 || !id.startsWith("AV"))
10        {
            throw new IncorrectIdException("The id should start from the letter AV
                because it is Active Virtual Game");
12        }

14    }
    public Equipment getVrEquipment()
16    {
        return vrEquipment;
18    }

20    public int calculatePrice(boolean peak) //checks peak time and type of Equipment
    {
22        if(peak == true)
        {
24            newPrice = price;
            return (int) newPrice;
26        }
        else if (peak == false && vrEquipment == Equipment.HEADSET_ONLY)
28        {
            newPrice = price * 0.9;
30            return (int) newPrice;
        }
        else if (peak == false && vrEquipment == Equipment.HEADSET_AND_CONTROLLER)
32        {
            newPrice = price * 0.95;
34            return (int) newPrice;
        }
        else
36        {
38            newPrice = price;
            return (int) newPrice;
40        }
    }

42    public String toString()
    {
44        //    return "Price of the game " + name + " with id " + id + " is: " + (int)
        newPrice + "p and equipment needed for this game: " + vrEquipment;
46        return "\n" + "\n" + "Name: " + name + "\n" + "Id: " + id + "\n" + "Price: " +
            price + "p \n" + "Equipment type: " + vrEquipment;
    }

48
    //    public static void main(String[] args)
50    //    {
    //        //Testing whether everything works fine
52    //        VirtualRealityGame vrgame = new VirtualRealityGame("VR boxing", "AV00000000
        ", 4, 10, Equipment.BODY_TRACKING_SUIT);
    //        VirtualRealityGame vrgame = new VirtualRealityGame("VR boxing", "AV00000000
        ", 4, 10, Equipment.HEADSET_ONLY);
54    //        //Testing whether discounts work fine
    //        System.out.println(vrgame.calculatePrice(true));

```

```
56 //      System.out.println(vrgame.calculatePrice(false));
//      System.out.println(vrgame);
58 //
//      //Testing whether there is sn exception related to the first letter
60 //      VirtualRealityGame vrgame1 = new VirtualRealityGame("VR boxing
", "0000000000", 4, 10, Equipment.BODY_TRACKING_SUIT);
//      //Testing related to the first two letters of the id because of VRGame class
62 //      VirtualRealityGame vrgame2 = new VirtualRealityGame("VR boxing", "A000000000
", 4, 10, Equipment.BODY_TRACKING_SUIT);
//      //Testing related to the number of characters
64 //      VirtualRealityGame vrgame3 = new VirtualRealityGame("VR boxing", "AV0000000000
", 4, 10, Equipment.BODY_TRACKING_SUIT);
//      }
66
68 }
```

Customer.java

```
public class Customer implements Comparable<Customer>{
2   private String customerID;
   private String customerName;
4   private int age;
   private LevelOfDiscount levelOfDiscount;
6   private int accBalance;

8   public Customer(String customerID,String customerName,int age,LevelOfDiscount
       levelOfDiscount)
   {
10      this.customerID = customerID;
       if(customerID.length() != 6)
12      {
           throw new InvalidCustomerException("Id should contain 6 characters!");
14      }
       this.customerName = customerName;
16      this.age = age;
       this.levelOfDiscount = levelOfDiscount;
18      accBalance = 0;
   }

20   public Customer (String customerID,String customerName,int accBalance,int age,
       LevelOfDiscount levelOfDiscount)
22   {
       this(customerID,customerName,age,levelOfDiscount);
24       if(levelOfDiscount != LevelOfDiscount.STUDENT && accBalance < 0)//throws
           exception if balance goes less than zero if it is not student
       {
26           throw new InvalidCustomerException("Account balance can't be zero!");
       }
       else if(customerID.length() != 6)//checks whether id has 6 characters
       {
30           throw new InvalidCustomerException("Id should contain 6 characters!");
       }
       else {
32           this.accBalance = accBalance;
34       }
       if(levelOfDiscount == LevelOfDiscount.STUDENT)//allows balance to go lower
           than zero if it is a student
       {
36           if(accBalance < -500)
38           {
               throw new InvalidCustomerException("Student's balance cannot be lower
                   than -500!");
40           }
               else
42           {
                   this.accBalance = accBalance;
44           }
               this.accBalance = accBalance;
46       }
   }

48   }

50   public String getCustomerID()
   {
52       return customerID;
   }
54   public String getCustomerName()
   {
56       return customerName;
```

```
    }
58    public int getAge()
    {
60        return age;
    }
62    public LevelOfDiscount getLevelOfDiscount()
    {
64        return levelOfDiscount;
    }
66    public int getAccBalance()
    {
68        return accBalance;
    }
70
72    public int addFunds(int amount)//adds money only if positive amount is entered
    {
74        if(amount<=0)
        {
76            System.out.println("Please enter the positive amount!");
78        }
        else
        {
80            return accBalance = accBalance + (amount);
82        }
        return accBalance;
84    }
86    public int chargeAccount(ArcadeGame game,boolean isPeak)//checked exception is
        thrown if balance doesn't have enough credits
    {
88        double totalCharge;
90        if(accBalance < game.getPrice())//
        {
92            try
            {
94                throw new InsufficientBalanceException("Your balance does not have
                    enough credits");
            }
            catch (InsufficientBalanceException e)
            {
96                System.out.println( e.getMessage());
98                return 0;
100            }
102        }
        if(game instanceof ActiveGame)//Also checks and throws checked exception if
            customer tries to play ActiveGame but they are too young
104        {
            if(age < ((ActiveGame) game).minAge)
            {
106                try
                {
108                    throw new AgeLimitException("You are too young!");
110                }
                catch (AgeLimitException e)
                {
112                    System.out.println( e.getMessage());
114                    return 0;
                }
            }
116        }
```

```

    }

118
    }
120    if(levelOfDiscount == LevelOfDiscount.STUDENT)//Gives discount based on the
        level of discount
    {
122        totalCharge = game.calculatePrice(isPeak) * 0.95;
        accBalance = accBalance - (int)totalCharge;
124        return (int)totalCharge;
    }
126    else if(levelOfDiscount == LevelOfDiscount.STAFF)
    {
128        totalCharge = game.calculatePrice(isPeak) * 0.90;
        accBalance = accBalance - (int)totalCharge;
130        return (int)totalCharge;
    }
132    else
    {
134        totalCharge = game.calculatePrice(isPeak);
        accBalance = accBalance - (int)totalCharge;
136        return (int)totalCharge;
    }
138

140
}

142    public int compareTo(Customer other)
    {
144        return Integer.compare(this.accBalance, other.accBalance); //Used to sort
        ArrayList of customers to determoine the richest
    }
146

148    public String toString()
    {
150
        return "\n" + "\n" + "Name: " + customerName + "\n" + "Id: " + customerID +
            "\n" + "Age: " + age + "\n" + "Level of discount: " + levelOfDiscount
            + "\n" + "Current balance: " + accBalance;
152    }
//    public static void main(String[] args)
154    {
        /// Customer customer = new Customer("123456", "Alex", 14, LevelOfDiscount.NONE
        , 5);
156    //    //chargeAccount methd works well when the lelevel of the discount is none
        /// Customer customer = new Customer("123456", "Alex", 14, LevelOfDiscount.
        STUDENT, 5);
158    //    //chargeAccount works well when level of the discount is STUDENT
        /// Customer customer = new Customer("123456", "Alex", 14, LevelOfDiscount.
        CMP_STAFF, 5);
160    //    //The same with CMP_STAFF
        /// Customer customer = new Customer("123456", "Alex", 8, LevelOfDiscount.
        CMP_STAFF, 5);
162    //    //Method has successfully given AgeLimitException
        /// Customer customer = new Customer("123456", "Alex", 14, LevelOfDiscount.
        CMP_STAFF, 1);
164    //    //Method has successfully given InsuficientBalanceException
        //    ActiveGame actGame = new ActiveGame("Game", "A002211345", 2, 10);
166    //    System.out.println(customer.chargeAccount(actGame, false));
        //
168    //    // addFunds method succesfully adds funds
        /// System.out.println(customer);
170    /// int newBalance = customer.addFunds(1);

```

```
172      //////      System.out.println(newBalance);
173
174      //      Customer cusstomer = new Customer("C45678", "Adam", 12, LevelOfDiscount.STUDENT);
175      toString testing
176      //      System.out.println(cusstomer.toString());
177      //
178      //      }
179      //
180      }
```


Arcade.java

```
2  import java.util.ArrayList;
   import java.util.Collections;
4
   public class Arcade {
6       private String arcadeName;
       private int revenue;
8       ArrayList<ArcadeGame> arcadeGamesCollection = new ArrayList<>();
       ArrayList<Customer> customers = new ArrayList<>();
10
       public Arcade (String arcadeName)
12       {
           this.arcadeName = arcadeName;
14       }

       public void addCustomer(Customer customer)
16       {
18           customers.add(customer);
           }

20       public void addArcadeGame(ArcadeGame game)
22       {
           arcadeGamesCollection.add(game);
24       }

       public String getArcadeName()
26       {
28           return arcadeName;
           }

30       public int getRevenue()
32       {
           return revenue;
34       }

36
       public Customer getCustomer(String customerID) throws InvalidCustomerException
38       //This method iterates through all clients and, if the provided id matches,
           returns the correct customer
       {
40           if(customerID.length() !=6 )
               {
42               throw new InvalidCustomerException("Id should have 6 characters");
               }
44           for(int i = 0; i<customers.size();i++)
               {
46               if(customers.get(i).getCustomerID().equals(customerID))
                   {
48                   return customers.get(i);
                   }
50               }
           System.out.println("There is no customer with provided id");
52           return null;

54       }

56       public ArcadeGame getArcadeGame(String gameID) throws IncorrectIdException
           //This method iterates through all games and, if the provided id matches, returns
           the correct game
       {
58           boolean exist = false;
```

```
60     for(int i=0; i<arcadeGamesCollection.size(); i++)
61     {
62         if(arcadeGamesCollection.get(i).getID().equals(gameID))
63         {
64             exist = true;
65             return arcadeGamesCollection.get(i);
66         }
67         if(gameID.length() !=10 && (!gameID.startsWith("C") || !gameID.startsWith
68             ("A")))
69         {
70             throw new IncorrectIdException("id should contain 10 characters and
71                 start from letter A or C depends on the game!");
72         }
73     }
74     if(!exist)
75     {
76         System.out.println("There is no game with that id");
77     }
78     return null;
79 }
80
81 public boolean processTransaction(String customerID,String gameID,boolean peak)
82 //this method gets game andd customer by their id and checks what type of the
83 //game is that
84 {
85     boolean wasUpdated = false;
86     Customer customer = getCustomer(customerID);
87     ArcadeGame game = getArcadeGame(gameID);
88     int balance = customer.getAccBalance();
89     int price = game.getPrice();
90     if(game instanceof ActiveGame)//if it is ActiveGame then it checks minimum
91         age and balance
92     {
93         if(balance >= price && customer.getAge() >= ((ActiveGame) game).minAge)
94         {
95             int charge = customer.chargeAccount(game,peak);
96             revenue = revenue + charge;
97             wasUpdated = true;
98         }
99     }
100     else
101     {
102         if(customer.getAge() < ((ActiveGame) game).minAge)
103         {
104             try
105             {
106                 throw new AgeLimitException("You are too young");
107             }
108             catch (AgeLimitException e)
109             {
110                 System.out.println(e.getMessage());
111             }
112         }
113         if(balance < price)
114         {
115             try
116             {
117                 String getName = getCustomer(customerID).getCustomerName();
118
```

```

        throw new InsufficientBalanceException(getName + ", you don't
            have enough credits");
120     }
122     catch (InsufficientBalanceException e)
124     {
        System.out.println(e.getMessage());
    }
126
128     }
130 }
132 else//If it is Cabinet game then it checks just a balance
134 {
136     if(balance >= price)
138     {
140         int charge = customer.chargeAccount(game,peak);
142         revenue = revenue + charge;
144         wasUpdated = true;
146     }
148     else
150     {
152         try
154         {
156             String getName = getCustomer(customerID).getCustomerName();
158             throw new InsufficientBalanceException(getName + ", you don't
160                 have enough credits");
162         }
164         catch (InsufficientBalanceException e)
166         {
168             System.out.println(e.getMessage());
170         }
172     }
174 }
176
178 return wasUpdated;
180 }

public Customer findRichestCustomer()
182 {
184     Collections.sort(customers);
186     // for(int i =0;i<customers.size();i++)
188     // {
190     //     System.out.println(customers.get(i)); Testing whether ArrayList is
192     // sorted
194     // }

196     Customer richest = customers.get(0);//checks and compares each customer with
198     the first one and assigns to the variable richest until the end of the
200     ArrayList
202     for(int i = 1;i<customers.size() ;i++)
204     {
206         if(customers.get(i).getAccBalance() > richest.getAccBalance())
208         {
210             richest = customers.get(i);
212         }
214     }
216
218     return richest;
220 }

```

```

178 public int getMedianGamePrice()
179 {
180     Collections.sort(arcadeGamesCollection);
181
182     if(arcadeGamesCollection.size() % 2 == 0) //checks whether ArrayList has even
        number of games if it is even then it finds two median ones and divides
        them by 2
    {
184         int numberOfElemnts = arcadeGamesCollection.size();
185         ArcadeGame median1 = arcadeGamesCollection.get(numberOfElemnts/2);
186         ArcadeGame median2 = arcadeGamesCollection.get(numberOfElemnts/2 -1);
187         int meddian = (median1.getPrice() + median2.getPrice()) / 2;
188         return meddian;
189     }
190     else //if it odd number of games then just find median
    {
192         ArcadeGame medianElement = arcadeGamesCollection.get(
            arcadeGamesCollection.size() / 2);
193         int median = medianElement.getPrice();
194         return median;
195     }
196 }
197
198 public int[] countArcadeGames()
199 { //This method simply checks the current type of each game and increments the
    count for that type until the end of the ArrayList is reached.
    int[] arcadeGames = new int[3];
    int cabinetG = 0;
    int activeG = 0;
    int vrG = 0;
    for(int i=0; i<arcadeGamesCollection.size(); i++)
    {
206         if(arcadeGamesCollection.get(i) instanceof CabinetGame)
207         {
208             cabinetG++;
209             arcadeGames[0] = cabinetG;
210         }
211         else if(arcadeGamesCollection.get(i) instanceof VirtualRealityGame)
212         {
213             vrG++;
214             arcadeGames[2] = vrG;
215         }
216         else
217         {
218             activeG++;
219             arcadeGames[1] = activeG;
220         }
221         if(i == arcadeGamesCollection.size() -1)
222         {
224             // System.out.println(arcadeGames[2]); used for testing to check how
                many games are there
                // System.out.println(arcadeGames[0]);
226             return arcadeGames;
227         }
228     }
229 }
230
231 return arcadeGames;
232 }
233
234

```

```

236     public static void printCorporateJargon()
    {
        System.out.println("GamesCo does not take responsibility for any accidents or
        fits of rage that occur on the premises");
    }

240     public String toString()
    {
242         return "Arcade name: " + arcadeName + "\n" + "Revenue: " + revenue + "\n" + "\n"
            + "Games list: " + arcadeGamesCollection + "\n" + "\n" + "List of
            customers: " + customers;
    }

244

246 //     public static void main(String[] args)
248 //     {
249 //         /////
250 //         Customer customer = new Customer("100000", "Bob", 600, 10, LevelOfDiscount
        .STAFF);
    //         ActiveGame game = new ActiveGame("great", "A100222333", 100, 10);
252 //         CabinetGame game1 = new CabinetGame("sss", "C100200300", 500, true);
    //         VirtualRealityGame game2 = new VirtualRealityGame("aaa", "AV10020033
        ", 400, 10, Equipment.BODY_TRACKING_SUIT);
254 //         VirtualRealityGame game3 = new VirtualRealityGame("aava", "AV10020033
        ", 6, 12, VirtualRealityGame.Equipment.BODY_TRACKING_SUIT);
    //         VirtualRealityGame game4 = new VirtualRealityGame("aaab", "AV10020033
        ", 9, 12, VirtualRealityGame.Equipment.BODY_TRACKING_SUIT);
256 //         VirtualRealityGame game5 = new VirtualRealityGame("aadab", "AV10020033
        ", 10, 12, VirtualRealityGame.Equipment.BODY_TRACKING_SUIT);
    //         Arcade arcade = new Arcade("Tennis");
258 //         arcade.addArcadeGame(game);
    //         arcade.addArcadeGame(game1);
260 //         arcade.addArcadeGame(game2);
    //         arcade.addArcadeGame(game3);
262 //         arcade.addArcadeGame(game4);
    //         arcade.addArcadeGame(game5);
264 //         arcade.getMedianGamePrice(); // Testing of the getMedianGamePrice
        succesfully sorts array base omn the price and returns median
    //         /////
266 //         arcade.countArcadeGames(); // Testing of the countArcadeGames ,it counts
        precise number of games and separate ActiveGames and VirtualRealityGames
    //         /////
268 //         arcade.addCustomer(customer);
    //         arcade.addArcadeGame(game); Testing of the addGame
    //         /////
272 //         Testing of the proccessTransaction.succesfully returns true if
        tracnsaction completed, false if not and stops transaction based on the exceptions
    //         Boolean transactionStatus = arcade.processTransaction(customer.
        getCustomerID(), game1.getID(), true);
274 //         System.out.println("Transaction status is : " + transactionStatus + "
        and current balance of the customer is " + customer.getAccBalance());
    //         arcade.customers.add(customer);
276 //         Customer str = arcade.getCustomer("100001");
    //         System.out.println(str);
278 //         /////
280 //         Testing whether findRichestCustomer displays actually the richest and it
        does
    //         Arcade arcade = new Arcade("Tennis");
282 //         Customer c1 = new Customer("123456", "Alex", 1, 20, Customer.
        LevelOfDiscount.CMP_STAFF);

```

```
//////// Customer c2 = new Customer("123458", "John", 2, 21, Customer.  
    LevelOfDiscount.CMP_STAFF);  
284 ////////// Customer c3 = new Customer("123457", "Bob", 3, 22, Customer.  
    LevelOfDiscount.CMP_STAFF);  
//////// Customer c4 = new Customer("123457", "Sara", 3, 22, Customer.  
    LevelOfDiscount.CMP_STAFF);  
286 ////////// Customer c5 = new Customer("123457", "Jess", 10, 22, Customer.  
    LevelOfDiscount.CMP_STAFF);  
//////// arcade.addCustomer(c1);  
288 ////////// arcade.addCustomer(c2);  
//////// arcade.addCustomer(c3);  
290 ////////// arcade.addCustomer(c4);  
//////// arcade.addCustomer(c5);  
292 ////////// arcade.findRichestCustomer();  
//  
294 //  
//    }  
296 }
```

AgeLimitException.java

```
public class AgeLimitException extends Exception {  
2     public AgeLimitException(String message) {  
        super(message);  
4     }  
}
```

Equipment.java

```
1 public enum Equipment {  
    HEADSET_ONLY, HEADSET_AND_CONTROLLER, BODY_TRACKING_SUIT  
3 }
```


IncorrectIdException.java

```
1 public class IncorrectIdException extends RuntimeException {
    public IncorrectIdException(String message)
3     {
        super(message);
5     }
    }
7
    // If I need to throw exception inside the constructor then it means
9 // I need to extend RuntimeException instead of Exception class because it should be
    unchecked exception
```

InsufficientBalanceException.java

```
1 public class InsufficientBalanceException extends Exception {  
    public InsufficientBalanceException(String message) {  
3        super(message);  
    }  
5 }
```

InvalidCustomerException.java

```
1 public class InvalidCustomerException extends RuntimeException {  
    public InvalidCustomerException(String message)  
3    {  
        super(message);  
5    }  
}
```

LevelOfDiscount.java

```
public enum LevelOfDiscount
2 {
    NONE,STAFF,STUDENT
4 }
```

Application

Compiler Invocation

```
javac -Xlint:unchecked -Xlint:deprecation -encoding UTF-8 -d
  prepasg741190476746829216classes Simulation.java ArcadeGame.java CabinetGame.java
  ActiveGame.java VirtualRealityGame.java Customer.java Arcade.java
  AgeLimitException.java Equipment.java IncorrectIdException.java
  InsufficientBalanceException.java InvalidCustomerException.java LevelOfDiscount.
  java
```

Compiler Messages

None.

Application Invocation

```
java Simulation
```

Messages to STDOUT

```
Amanda Pham, you have successfully paid for the game. Have fun!
Amber Y Basse, you have successfully paid for the game. Have fun!
Kiri Kramer, you have successfully paid for the game. Have fun!
Amanda Pham, you don't have enough credits
Charis Jaramillo, you have successfully paid for the game. Have fun!
Aman Crosby, you have successfully paid for the game. Have fun!
Aoife Crouch, you have successfully paid for the game. Have fun!
Dom Seeker, you have successfully paid for the game. Have fun!
Customer has successfully been added to the system
Paige Barclay, you have successfully paid for the game. Have fun!
Dom Seeker, you don't have enough credits
Paige Barclay, you don't have enough credits
Charles James-Warne, you don't have enough credits
Lucille Swan, you have successfully paid for the game. Have fun!
You are too young
Charles James-Warne, you don't have enough credits
Amber Y Basse, you have successfully paid for the game. Have fun!
Lucille Swan, you have successfully paid for the game. Have fun!
Lucille Swan, you have successfully paid for the game. Have fun!
Customer has successfully been added to the system
Amber Y Basse, you have successfully paid for the game. Have fun!
Dom Seeker, you don't have enough credits
Aoife Crouch, you don't have enough credits
Cat Chives-Keller, you have successfully paid for the game. Have fun!
Dom Seeker, you have successfully paid for the game. Have fun!
Charis Jaramillo, you have successfully paid for the game. Have fun!
Charis Jaramillo, you have successfully paid for the game. Have fun!
Cat Chives-Keller, you don't have enough credits
Customer has successfully been added to the system
Wrong format of id: id should contain 10 characters and start from letter A or C
  depends on the game!
Aoife Crouch, you have successfully paid for the game. Have fun!
You have successfully added money to your balance
Dom Seeker, you have successfully paid for the game. Have fun!
Paige Barclay, you have successfully paid for the game. Have fun!
Amanda Pham, you don't have enough credits
You have successfully added money to your balance
Amanda Pham, you have successfully paid for the game. Have fun!
Paige Barclay, you don't have enough credits
There is no customer with provided id
Failed to add funds
You have successfully added money to your balance
```

Paige Barclay, you have successfully paid for the game. Have fun!
Dom Seeker, you don't have enough credits
Dom Seeker, you don't have enough credits
Aman Crosby, you have successfully paid for the game. Have fun!
You have successfully added money to your balance
Amanda Pham, you have successfully paid for the game. Have fun!
Aman Crosby, you don't have enough credits
Amanda Pham, you have successfully paid for the game. Have fun!
Amanda Pham, you have successfully paid for the game. Have fun!
You have successfully added money to your balance
Dom Seeker, you have successfully paid for the game. Have fun!
Dom Seeker, you have successfully paid for the game. Have fun!
Charis Jaramillo, you have successfully paid for the game. Have fun!
Katriona Blownes, you have successfully paid for the game. Have fun!
Charis Jaramillo, you have successfully paid for the game. Have fun!
Customer has successfully been added to the system
Paige Barclay, you don't have enough credits
Lucille Swan, you have successfully paid for the game. Have fun!
Kiri Kramer, you have successfully paid for the game. Have fun!
Kiri Kramer, you have successfully paid for the game. Have fun!
Aoife Crouch, you don't have enough credits
Paige Barclay, you have successfully paid for the game. Have fun!
Kiri Kramer, you have successfully paid for the game. Have fun!
Charis Jaramillo, you have successfully paid for the game. Have fun!
Kiri Kramer, you have successfully paid for the game. Have fun!

The richest customer is

Name: Mantis Toboggan
Id: B473Z4
Age: 72
Level of discount: STAFF
Current balance: 20100

The median game price is: 200

Cabinet/Active/Virtual games: [7, 8, 7]
GamesCo does not take responsibility for any accidents or fits of rage that occur on the premises

Arcade name: Arcade of fun
Revenue: 8775

Games list: [

Name: "Virtual UEA Tour"
Id: AVI1USPBNG
Price: 0p
Equipment type: HEADSET_ONLY,

Name: "Reaction Test 2000"
Id: CBGCR27FQM
Price: 40p
Status of the reward: : true,

Name: "D'Arcy Thompson Pinball"
Id: CPVOHUHOZH
Price: 50p
Status of the reward: : true,

Name: "Foosball"
Id: AHWOHK1F03
Price: 80p

Minimum age:3,

Name: "American Pool"

Id: A4FJTZLIVA

Price: 100p

Minimum age:12,

Name: "CMP Chomp Man"

Id: C7S6SYBL8R

Price: 120p

Status of the reward: : false,

Name: "Snooker"

Id: AJFS153KZV

Price: 120p

Minimum age:12,

Name: "Street Wrestler V"

Id: CQLS3YESOM

Price: 140p

Status of the reward: : false,

Name: "Air Hockey"

Id: AD65E3UJQJ

Price: 180p

Minimum age:12,

Name: "Darts"

Id: AX5YNVUJA9

Price: 200p

Minimum age:16,

Name: "Plumber Kart 8"

Id: CNQZPI7G5E

Price: 200p

Status of the reward: : false,

Name: "Plonky Kong"

Id: CAPSD7TLC6

Price: 200p

Status of the reward: : false,

Name: "Clock Crisis"

Id: CXPVC0DBXU

Price: 220p

Status of the reward: : true,

Name: "Fly Like a Seagull!"

Id: AV55GWU6PS

Price: 350p

Equipment type: HEADSET_ONLY,

Name: "Virtual Petting Zoo"

Id: AVLD1ZDNXE

Price: 400p

Equipment type: HEADSET_AND_CONTROLLER,

Name: "Snowball Fight"

Id: AVX5KN5T30

Price: 400p

Equipment type: BODY_TRACKING_SUIT,

Name: "Football Pool"

Id: AMURG8FXMK

Price: 500p

Minimum age:3,

Name: "Pickaxe Crafting Simulator"

Id: AVP09GJA1Z

Price: 500p

Equipment type: HEADSET_AND_CONTROLLER,

Name: "Dance like a Professor"

Id: AVSKVMRB9U

Price: 800p

Equipment type: BODY_TRACKING_SUIT,

Name: "Beer Pong"

Id: AL2ETWHG0Q

Price: 1000p

Minimum age:18,

Name: "VR Paintball"

Id: AV90PS1LRT

Price: 1000p

Equipment type: BODY_TRACKING_SUIT,

Name: "Axe Throwing"

Id: AN234FQD9D

Price: 2000p

Minimum age:18]

List of customers: [

Name: Barry Guy

Id: 1C6498

Age: 17

Level of discount: NONE

Current balance: 0,

Name: Charles James-Warne

Id: 51CX13

Age: 12

Level of discount: NONE

Current balance: 10,

Name: Jacob Peraltera

Id: P54578

Age: 32

Level of discount: STUDENT

Current balance: 10,

Name: Amanda Pham

Id: A64248

Age: 45

Level of discount: NONE

Current balance: 40,

Name: Edwin Shea

Id: 144374

Age: 24

Level of discount: NONE

Current balance: 100,

Name: Penelope Nootly

Id: 800813

Age: 5
Level of discount: NONE
Current balance: 100,

Name: Dom Seeker
Id: 174450
Age: 19
Level of discount: STUDENT
Current balance: 108,

Name: Aman Crosby
Id: 546R82
Age: 62
Level of discount: STAFF
Current balance: 161,

Name: Lemmington Steele
Id: 889027
Age: 18
Level of discount: STUDENT
Current balance: 200,

Name: Jorge Pimento
Id: 985F83
Age: 23
Level of discount: STUDENT
Current balance: 200,

Name: Katriona Blownes
Id: 681212
Age: 21
Level of discount: NONE
Current balance: 200,

Name: Irene Freeman
Id: 387036
Age: 10
Level of discount: NONE
Current balance: 240,

Name: Aurora Mcleod
Id: 9F0610
Age: 39
Level of discount: STAFF
Current balance: 260,

Name: Paige Barclay
Id: 901420
Age: 29
Level of discount: NONE
Current balance: 276,

Name: Cat Chives-Keller
Id: 506VX5
Age: 15
Level of discount: NONE
Current balance: 280,

Name: Lois Romero
Id: 203685
Age: 41
Level of discount: STAFF
Current balance: 360,

Name: John-Paul Clay
Id: 58R526
Age: 47
Level of discount: STAFF
Current balance: 400,

Name: Maaria Crossley
Id: 555765
Age: 39
Level of discount: STAFF
Current balance: 400,

Name: Charis Jaramillo
Id: 518370
Age: 19
Level of discount: STUDENT
Current balance: 558,

Name: Lucille Swan
Id: 748A66
Age: 31
Level of discount: STAFF
Current balance: 602,

Name: Thea Worthington
Id: SGRQ01
Age: 17
Level of discount: NONE
Current balance: 800,

Name: Roshni Stephens
Id: 117DA4
Age: 15
Level of discount: NONE
Current balance: 1000,

Name: Moses Rivers
Id: 54A0N3
Age: 20
Level of discount: STUDENT
Current balance: 1000,

Name: Khloe Stokes
Id: 522136
Age: 6
Level of discount: NONE
Current balance: 1000,

Name: Aoife Crouch
Id: 738164
Age: 29
Level of discount: NONE
Current balance: 1000,

Name: Jimmy Baker
Id: 973SA2
Age: 4
Level of discount: NONE
Current balance: 1000,

Name: Amber Y Basse
Id: 6048D1

Age: 37
Level of discount: STUDENT
Current balance: 1020,

Name: Nour Buxton
Id: 343172
Age: 5
Level of discount: NONE
Current balance: 1200,

Name: Rowena Kaiser
Id: 264474
Age: 19
Level of discount: STUDENT
Current balance: 1600,

Name: Kiri Kramer
Id: 2612CX
Age: 38
Level of discount: NONE
Current balance: 2400,

Name: Lewie King
Id: 305459
Age: 47
Level of discount: NONE
Current balance: 3200,

Name: Amy Mckinney
Id: 3S6965
Age: 17
Level of discount: NONE
Current balance: 4000,

Name: Gravin Clawley
Id: 18761S
Age: 7
Level of discount: NONE
Current balance: 5000,

Name: Mantis Toboggan
Id: B473Z4
Age: 72
Level of discount: STAFF
Current balance: 20100]

Messages to STDERR

None.